

FamilyGraph Explorer 🌳 (Python Version)

An innovative, interactive web application for visualizing and exploring family relationships stored in a Neo4j graph database. Built with **Python Flask**, **React**, **D3.js**, and **Neo4j**.

Show Image

Show Image

Show Image

🚀 Quick Start (Python)

1. Install Python Dependencies

```
bash

# Create virtual environment (recommended)
python -m venv venv

# Activate virtual environment
# On Windows:
venv\Scripts\activate
# On macOS/Linux:
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt
```

2. Configure Neo4j Connection

Create a `.env` file in the root directory:

```
env
```

```
NEO4J_URI=bolt://localhost:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=your_password_here
PORT=5000
```

3. Verify Your Neo4j Database

Make sure your Neo4j database is running and contains Person nodes:

```
bash

# Check if Neo4j is running
neo4j status

# Start Neo4j if not running
neo4j start
```

4. Start the Python Flask Server

```
bash

python app.py
```

You should see:

```
 FamilyGraph Explorer - Python Backend
 Server running on: http://localhost:5000
 Neo4j URI: bolt://localhost:7687
```

5. Update Frontend to Use Python Backend

Open `family-tree-production.html` and change line 29:

```
javascript

// Change from:
const API_BASE_URL = 'http://localhost:3000/api';

// To:
const API_BASE_URL = 'http://localhost:5000/api';
```

6. Open the Application

Open `family-tree-production.html` in your browser!

Project Structure

```
familygraph-explorer/
├── app.py                # Python Flask backend
├── requirements.txt      # Python dependencies
├── family-tree-production.html # React frontend
├── .env                 # Environment configuration
└── README-PYTHON.md    # This file
```

Python Backend Features

Flask Application (`app.py`)

Key Features:

-  RESTful API design
-  Neo4j Python driver integration
-  CORS enabled for frontend access
-  Comprehensive error handling
-  Logging for debugging
-  Environment variable configuration

API Endpoints:

Method	Endpoint	Description
GET	/health	Health check and Neo4j connection status
GET	/api/persons	Get all persons
GET	/api/persons/<id>	Get person by ID
POST	/api/persons	Create new person
PUT	/api/persons/<id>	Update person
DELETE	/api/persons/<id>	Delete person
GET	/api/relationships	Get all relationships
GET	/api/persons/<id>/relationships	Get person's relationships
POST	/api/relationships	Create new relationship
GET	/api/path/<id1>/<id2>	Find shortest path
GET	/api/stats	Get database statistics
GET	/api/search?query=name	Search persons
GET	/api/generations	Get root generation

Testing the API with Python

You can test the API using Python's `requests` library:

```
python
```

```
import requests

# Get all persons
response = requests.get('http://localhost:5000/api/persons')
persons = response.json()
print(persons)

# Create a new person
new_person = {
    'name': 'John Doe',
    'nickName': 'Johnny',
    'gender': 'male',
    'dateOfBirth': '1990-05-15'
}
response = requests.post('http://localhost:5000/api/persons', json=new_person)
print(response.json())

# Find path between two persons
response = requests.get('http://localhost:5000/api/path/1/5')
path = response.json()
print(path)
```

Working with Neo4j in Python

Basic Neo4j Queries

```
python
```

```
from neo4j import GraphDatabase

# Connect to Neo4j
driver = GraphDatabase.driver(
    'bolt://localhost:7687',
    auth=('neo4j', 'your_password')
)

# Get all persons
with driver.session() as session:
    result = session.run("MATCH (p:Person) RETURN p")
    for record in result:
        person = record['p']
        print(f"Name: {person['name']}, Gender: {person['gender']}")

# Find descendants
with driver.session() as session:
    result = session.run("""
        MATCH (ancestor:Person {name: 'Robert Anderson'})-[:has_child*]->(descendant:Person)
        RETURN descendant.name as name
    """)
    for record in result:
        print(f"Descendant: {record['name']}")

# Close driver
driver.close()
```



Data Import Script

Create a script to populate your database:

```
python
```

```

# import_family_data.py
from neo4j import GraphDatabase

driver = GraphDatabase.driver(
    'bolt://localhost:7687',
    auth=('neo4j', 'your_password')
)

def import_family():
    with driver.session() as session:
        # Clear existing data (optional)
        session.run("MATCH (n:Person) DETACH DELETE n")

        # Create persons
        persons = [
            {'name': 'Robert Anderson', 'nickName': 'Bob', 'gender': 'male', 'dateOfBirth': '1945-03-15'},
            {'name': 'Margaret Anderson', 'nickName': 'Maggie', 'gender': 'female', 'dateOfBirth': '1947-07-22'},
            {'name': 'James Anderson', 'nickName': 'Jim', 'gender': 'male', 'dateOfBirth': '1970-05-10'},
        ]

        for person in persons:
            session.run("""
                CREATE (p:Person {
                    name: $name,
                    nickName: $nickName,
                    gender: $gender,
                    dateOfBirth: $dateOfBirth
                })
            """, **person)

        # Create relationships
        session.run("""
            MATCH (robert:Person {name: 'Robert Anderson'})
            MATCH (margaret:Person {name: 'Margaret Anderson'})
            MATCH (james:Person {name: 'James Anderson'})
            CREATE (robert)-[:spouse_of]->(margaret)
            CREATE (robert)-[:has_child]->(james)
            CREATE (margaret)-[:has_child]->(james)
        """)

        print("✅ Family data imported successfully!")

if __name__ == '__main__':
    import_family()
    driver.close()

```

Run it:

```
bash

python import_family_data.py
```



Debugging Tips

Check if Flask is running:

```
bash

curl http://localhost:5000/health
```

View Flask logs:

The app runs with `debug=True`, so you'll see detailed logs in the terminal.

Test Neo4j connection:

```
python

from neo4j import GraphDatabase

driver = GraphDatabase.driver('bolt://localhost:7687', auth=('neo4j', 'password'))

try:
    with driver.session() as session:
        result = session.run("RETURN 1")
        print("✅ Connected!")
except Exception as e:
    print(f"❌ Error: {e}")
finally:
    driver.close()
```



Deployment Options

1. Development Server (Current)

```
bash

python app.py
```

2. Production with Gunicorn

```
bash

gunicorn -w 4 -b 0.0.0.0:5000 app:app
```


3. Docker Deployment

```
dockerfile

FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

EXPOSE 5000
CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:5000", "app:app"]
```

Build and run:

```
bash

docker build -t familygraph .
docker run -p 5000:5000 --env-file .env familygraph
```



Security Best Practices

1. **Never commit** `.env` file - Add to `.gitignore`
2. **Use strong passwords** for Neo4j
3. **Enable authentication** in production
4. **Configure CORS properly** - Restrict to your frontend domain
5. **Use HTTPS** in production

Example production CORS setup:

```
python

from flask_cors import CORS

# Only allow requests from your frontend domain
CORS(app, resources={
    r"/api/*": {
        "origins": ["https://yourdomain.com"]
    }
})
```

Useful Neo4j Queries

Find all childless persons:

```
cypher

MATCH (p:Person)
WHERE NOT (p)-[:has_child]->()
RETURN p.name, p.dateOfBirth
```

Find siblings:

```
cypher

MATCH (p1:Person)<-[:has_child]-(parent)-[:has_child]->(p2:Person)
WHERE ID(p1) < ID(p2)
RETURN p1.name as person1, p2.name as person2, parent.name as parent
```

Count descendants by person:

```
cypher

MATCH (p:Person)-[:has_child*]->(descendant)
RETURN p.name, count(descendant) as descendantCount
ORDER BY descendantCount DESC
```

Frontend Customization

The frontend connects via API, so you can:

1. Build a custom React app
2. Use Vue.js or Angular
3. Create a mobile app
4. Build a desktop application

Just point to: `http://localhost:5000/api`

Additional Python Packages

Install optional packages for enhanced functionality:

```
bash
```

```
# Data validation
```

```
pip install marshmallow
```

```
# API documentation
```

```
pip install flask-swagger-ui
```

```
# Testing
```

```
pip install pytest pytest-flask
```

```
# Database migrations
```

```
pip install alembic
```

Contributing

Feel free to:

- Add new API endpoints
- Improve error handling
- Add data validation
- Create tests
- Optimize queries



License

MIT License - Use freely for personal or commercial projects.

Built with  Python for family historians!